PHASE 00 / 21

# Architecture & infrastructure

Poser le socle technique reproductible : backend, frontend, base, tâches, déploiement.

| | |
|---|---|
| **Version** | MVP |
| **Complexité** | Moyenne |
| **Statut actuel** | Environ 90 % (reste CI/CD, sauvegardes) |
| **Prérequis** | Aucun |

## Modules couverts

- Transverse (aucun module métier)

## Étapes backend (dans cet ordre)

1. README racine : prérequis, lancement avec et sans Docker, commandes de test
2. requirements-dev.txt séparé (outils de test et de lint)
3. CI GitHub Actions : tests Django, ruff, lint et build du frontend
4. Production : gunicorn + whitenoise (ou nginx), Dockerfiles backend et frontend, healthcheck sur /api/v1/health/
5. Sauvegarde PostgreSQL (pg_dump quotidien, rotation) et restauration testée une fois
6. Logs structurés et suivi d'erreurs (Sentry, optionnel)
7. Vérifier qu'aucun secret n'est dans l'historique git

## Frontend web (Next.js)

- Composants UI communs (Button, Input, Card, Badge) dans components/ui
- AppShell réutilisable (sidebar + en-tête)
- Client d'API central (lib/api) avec gestion d'erreurs
- ESLint et Prettier
- Supprimer les définitions dupliquées de .material-symbols-outlined

## Application mobile (Expo)

- Décider de la structure : monorepo avec dossier mobile/ (Expo)
- Aucun code à ce stade

## Tests à écrire

- La CI échoue sur un test cassé
- docker compose up démarre db, redis, backend, worker, beat, frontend

## Critère de sortie de la phase

[ ] Un nouveau développeur lance le projet en suivant uniquement le README

[ ] La CI passe au vert sur la branche main

## Pièges à éviter

! Ne pas figer trop tôt les composants UI : les enrichir au fil des phases

! Ne jamais committer le fichier .env